Final Project: Document Assistant Prototype for Antitrust/Regulatory Cases

Welcome to the Final Project for the Large Language Models (LLM) Course.

Project Objective

In this project, you will work in groups to develop a **Document Assistant Prototype** tailored for a specific and well-defined use case, for instance antitrust or regulatory compliance (or anything else that the group finds interesting and useful). You will build a **Retrieval-Augmented Generation (RAG)** pipeline capable of answering user queries based on a specific corpus of documents.

The main goals of this project are:

1. **Ingest and Chunk** a custom corpus of documents, collected by the group to cover the specific use case you chose.
2. **Build a Retriever** that supports:
   - **Vector Search** (semantic lookup using either local embeddings or API calls to an OpenAI-compatible embedding model).
   - **Keyword Search** (exact matching using the BM25 algorithm, using the provided implementation).
3. **Build the Generation Loop** that constructs a prompt containing the query and retrieved context, and sends it to the LLM.
4. **Compare the RAG pipeline** against a **Direct LLM Prompting** baseline (i.e. prompting the model without context collected by retrieving chunks from your custom documents corpus) and evaluate the difference.
5. **Experiment** with different ways to improve the performance of the system, beyond the simple naive RAG implementation.
6. **Report** your journey and your findings in a final document.

About the Default Corpus

We provide a small example corpus made of three public EU competition regulation documents from EUR-Lex:

- *The EU Merger Regulation (EC 139/2004)*
- *The Horizontal Merger Guidelines*
- *The Non-Horizontal Merger Guidelines*

> ***This default corpus is intended only as a starting point and as an example.*** *Groups are strongly encouraged to* ***specialize their project*** *into a domain or a specific use case of their choosing by replacing or extending the corpus with their own documents. For example, you could focus on a specific set of EU merger decisions for a particular industry, national antitrust case law, data protection regulations, or any other document-heavy domain that interests your group.*

The default corpus can be downloaded from the course platform into the `data/` folder.

To use your own corpus, simply place your PDF files in the `data/` directory and remove the sample ones.

IMPORTANT: only use documents that are either already in the public domain (strongly encouraged) or that do not have any data privacy limitation, as you will send parts of them to LLM models hosted by third party providers. Neither the instructors nor anyone else involved in the course holds any responbility on how you use and share the documents you select.

Folder Structure

```
final_project/
  ├── README.md                  # This document, containing all the project instructio
  ns
  ├── README.pdf                 # Same content as this file, just in PDF format
  ├── document_assistant.ipynb   # Main interactive notebook (your workspace)
  ├── utils.py                   # Utility functions (provided and ready to use as-is,
  can be customized if needed)
  ├── requirements.txt           # Project dependency list (can be extended as needed)
  ├── .env.example               # Template for API keys and model settings
  └── data/                      # PDF document corpus
```

About `utils.py`

The `utils.py` module contains ready-to-use utility functions for PDF loading, search, embeddings, and LLM interaction. **These are provided as a simplified starting point**: you don't need to modify them to complete the project, but you are free to customize, extend, or replace any of them. See the `docu ment_assistant.ipynb` notebook for detailed documentation of each function.

Setup Instructions

The following commands assume you have a terminal open within the project folder. On VSCode, you can open one by clicking on "Terminal > New Terminal" in the command bar at the top; on Google Colab, there is a "Terminal" button in the lower left corner of the interface. If you need to revise how to setup a python project, you can go back to the first practice session (1_1_05 - Getting started).

1. Package Management with `uv`

As shown in the course exercises, we recommend using `uv` as a package manager for speed and reproducibility:

```
uv venv
  uv pip install -r requirements.txt
```

2. Configure Environment Variables

Copy the provided example env file:

```
cp .env.example .env
```

Open `.env` and configure **one** of the three available options for accessing an LLM (supporting both cloud providers if you have access to them, or local inference with open models):

- **Option A — Azure OpenAI**: Set `AZURE_OPENAI_API_KEY` and `AZURE_OPENAI_ENDPOINT`.
- **Option B — OpenAI API / Local Server**: Set `OPENAI_API_KEY` (and optionally `OPENAI_BASE_URL` for local servers like vLLM/Ollama or for using the instructor-provided api keys).
- **Option C — Local Hugging Face Model**: Set `LOCAL_MODEL_NAME` to load a model directly with the `transformers` library (see section below).

3. Open the Notebook

If you use an IDE that natively supports notebooks files (e.g., VSCode) you can simply open the `document_assistant.ipynb` file in the IDE, otherwise you can view it in a browser running the following code:

```
uv run jupyter notebook document_assistant.ipynb
```

Running Local Models

NOTE: if you do not want to handle the complexity of running models locally or plan to use either your own api key or the one provided by the instructors, you can safely skip this section.

If you don't have access to an OpenAI or Azure API, you can run models locally using the Hugging Face `transformers` library. The `create_llm_client` function provided in `utils.py` handles model loading and inference behind a unified interface — you just set the model name in your `.env` file.

Suggested Models

In general, any model from the HuggingFace Hub should work, just copy-paste their name in the standard `ORGANIZATION/MODEL_NAME` format. Below you find some suggestions from us.

Model	Parameters	VRAM Required	Notes
`HuggingFaceTB/SmolLM2-135M-Instruct`	135M	~400 MB	Extremely small, great for CPU usage. Lower quality than larger models.
`HuggingFaceTB/SmolLM2-1.7B-Instruct`	1.7B	~4 GB	Small and fast. Good for testing on CPU (will take a while, up to a few minutes per query) or low-end GPUs.
`Qwen/Qwen3-4B`	4B	~8 GB (FP16)	Excellent quality for its size. Supports thinking and non-thinking modes. Needs a GPU for generating text at interactive speed.

Hardware Options

- **Google Colab (Free Tier)**: Provides a Tesla T4 GPU with 16 GB VRAM. This is sufficient for both models above in FP16 precision. Just be sure to select *Runtime → Change runtime type → T4 GPU*.
- **Personal GPU**: Any GPU with ≥8 GB VRAM (e.g. RTX 3060, RTX 4060) can comfortably run the 4B model in FP16.
- **CPU-Only**: Possible for the smallest models (up to ~1-2B parameters). Inference will be slow (expect at least 10-30 seconds per response) but functional for development and testing. The `utils.py` will automatically use `float32` and run on the CPU when no GPU is detected.

Configuration

To use a local model, set the `LOCAL_MODEL_NAME` environment variables to one of the suggested models (or a different one, if you want to try) in your `.env` file.

> ***Tip**: If your GPU doesn't have enough memory, you can try quantization with the `bitsandbytes` library (4-bit). To enable this, you would need to modify the `TransformersClient` class in `utils.py` to pass a `BitsAndBytesConfig` — this is an optional advanced exercise.*

Project Requirements & Deliverables

At the end of the project (October 24) you are espected to deliver the following, by uploading into the course web platform:

1. Code

Your completed `document_assistant.ipynb` notebook, which must contain functional implementations of:

- **Chunking Strategy**: How you split your documents into retrievable units.
- **Retrieval Engine**: Using both BM25 keyword search and semantic vector search.
- **RAG Generation Loop**: Prompt template design and LLM generation using retrieved context.
- **Evaluation Walkthrough**: A comparison of answers generated by the **RAG Pipeline** vs. the **Base LLM (Direct Prompting)**, explaining why the RAG answers are more accurate or grounded.
- **Additional Experimentation**: The implementation and evaluation of any extra improvement that you worked on.

2. Report

A final report document for the project, including:

- A high-level explanation of your final implementation.
- A high-level explanation of the evaluation methods you used.
- The list of improvements you implemented, together with the impact on the performance of the RAG pipeline.
- Some examples of the system answering domain-specific queries based on your document corpus, together with an evaluation of them (include at least one success and one failure examples).

Evaluation Criteria

The project will be evaluated on the following dimensions:

- **Use Case**: How relevant and impactful is the use case you selected, and how well the selected document corpus supports it.
- **Experimentation Breadth**: How many different improvements were implemented and tested on top of the base RAG pipeline.
- **Performance Improvement**: How much better does the final version of the RAG system perform compared to the base RAG pipeline.
- **Code Clarity & Documentation**: How easy to follow and well-structured is the final code, and how well does the final report describe the final RAG system and the results.